
LoopVLA: Learning Sufficiency in Recurrent Refinement for Vision-Language-Action Models

Boyang Shen¹, Kaixiang Yang¹, Hao Wang¹, Qiuyu Yu², Qiang Xie², Qiang Li^{1,*}, Zhiwei Wang^{1,*}

¹Huazhong University of Science and Technology

²Wuhan United Imaging Surgical Co.,Ltd. (UIS)

*Corresponding authors:{liqiang8,zwwang}@hust.edu.cn

Abstract

Current Vision-Language-Action (VLA) models typically treat the deepest representation of a vision-language backbone as universally optimal for action prediction. However, robotic manipulation is composed of many frequent closed-loop spatial adjustments, for which excessive abstraction may waste computation and weaken low-level geometric cues essential for precise control. Existing early-exit strategies attempt to reduce computation by stopping at predefined layers or applying heuristic rules such as action consistency, but they do not directly answer when a representation is actually sufficient for action. In this paper, we present **LoopVLA**, a recurrent VLA architecture that jointly learns representation refinement, action prediction, and sufficiency estimation. LoopVLA iteratively applies a shared Transformer block to refine multimodal tokens, and at each iteration produces both a candidate action and a sufficiency score that estimates whether further refinement is necessary. By sharing parameters across iterations, LoopVLA decouples refinement from absolute layer indices and grounds sufficiency estimation in the evolving representation itself. Since sufficiency has no direct supervision, we introduce a self-supervised distribution alignment objective, where intermediate confidence scores are trained to match the relative action quality across refinement steps, thereby linking sufficiency learning to policy optimization signals. Experiments on LIBERO, LIBERO-Plus, and VLA-Arena show that LoopVLA pushes the efficiency-performance frontier of VLA policies, reducing parameters by 45% and improving inference throughput by up to 1.7 times while matching or outperforming strong baselines in task success.

1 Introduction

Vision-Language-Action (VLA) models represent a significant step toward generalist robotic manipulation, using large-scale multimodal pretraining to bridge perceptual understanding and physical interaction [1, 2, 3, 4]. A typical VLA system builds upon a vision-language model (VLM) to encode visual observations and language instructions into latent representations, and then employs an action head that maps these representations to continuous control outputs [5, 6, 7, 8, 9]. Most existing designs follow a *late-output* paradigm, where only the final-layer representation of the VLM is used for action prediction.

Despite its simplicity, the late-output paradigm implicitly assumes that the deepest, most abstract representation is universally suitable for all action decisions. This assumption, however, is not well aligned with the heterogeneous nature of embodied decision-making [10, 11]. In robotic manipulation, actions vary substantially from simple, reactive adjustments to complex, fine-grained manipulations. The latter may demand richer semantic abstraction, whereas the former depends more directly on low-level spatial, geometric, and reactive cues.

This observation motivates a key problem in VLA models: *representational sufficiency*. For action prediction, the objective is not necessarily to reach the deepest representation, but to obtain a representation that is *sufficient* for the current control decision. An under-refined representation may lead to inaccurate actions, while an over-processed representation may dilute task-relevant cues and reduce inference efficiency. This perspective also aligns with biological motor control, where simple reflexive responses rely on short neural pathways, while complex behaviors engage deeper processing circuits [10, 11].

Recent efforts have begun to exploit intermediate representations to overcome the rigidity of the late-output paradigm. These methods can be broadly organized into two categories. The first category requires a full forward pass through the VLM and then performs *post-hoc aggregation or selection* of features from multiple layers via auxiliary mechanisms [12, 13], as illustrated in Fig. 1(a). Although such methods can harness multi-level information beyond the final layer, they treat representation selection as a decoupled, retrospective step and invariably incur the full computational cost of processing all layers.

The second category relies on *early-exit* strategies that stop computation before the final layer. Static approaches [14, 15] directly use a predefined intermediate layer, remaining inherently rigid to varying task demands. Dynamic approaches, by contrast, allow the model to exit at different depths depending on the input. However, they typically rely on heuristic rules such as action consistency [16], which are not explicitly aligned with action quality and may not reliably indicate representational sufficiency.

In this work, we argue that representational sufficiency should be treated as an *intrinsic property* of the policy itself. This calls for a unified formulation in which representation refinement and action prediction are jointly modeled, enabling the policy to dynamically determine how much computation is needed.

To this end, we propose **LoopVLA**, a VLA architecture that models representation refinement and sufficiency *in situ* through a recurrent formulation. Specifically, we introduce a dual-head design: at each processing stage, an action head produces a candidate control output, while a gating head estimates a confidence score that quantifies the sufficiency of the current representation for the action at hand. This allows sufficiency to be explicitly modeled and directly tied to decision quality.

A straightforward implementation might attach such heads to each layer of the current VLM. However, this risks introducing biases tied to absolute layer indices, rather than grounding sufficiency in representational content, as illustrated in Fig. 1(b). To avoid this, we adopt a recurrent parameter-sharing design: a shallow Transformer model is applied iteratively, and the two heads are attached to each iteration, as illustrated in Fig. 1(c). This loop structure decouples refinement from absolute depth, compelling the model to base sufficiency estimates on the evolving features themselves.

As no ground-truth labels exist for representational sufficiency, we learn it through distribution alignment. During training, a fixed number of iterations is performed, and both the negative action losses and the confidence scores are normalized across iterations to form probability distributions. The confidence distribution is then trained to match the loss-derived distribution via a cross-entropy objective. This formulation treats the loss profile as soft supervision, enabling the model to recognize sufficient representations without any external annotation.

The main contributions of this work are summarized as follows:

1. Building upon prior representational sufficiency studies, we reformulate VLA representation learning as an iterative refinement process with a shared recurrent Transformer, and introduce a distribution alignment objective for learning sufficiency estimation from action optimization signals.
2. We propose LoopVLA, a recurrent VLA architecture that replaces the conventional fixed-depth Transformer with a shared looped Transformer for progressive refinement. The proposed design reduces model size by **45%** while maintaining strong performance.
3. We introduce a dual-head design with an action head and a sufficiency head, enabling sufficiency estimation directly from evolving representations rather than heuristic early-exit criteria.
4. We evaluate LoopVLA on LIBERO, LIBERO-Plus, and VLA-Arena, demonstrating improved inference efficiency while maintaining or improving policy performance over strong baselines, including up to **1.7 $\times$** higher inference throughput in extreme settings with substantially fewer parameters.

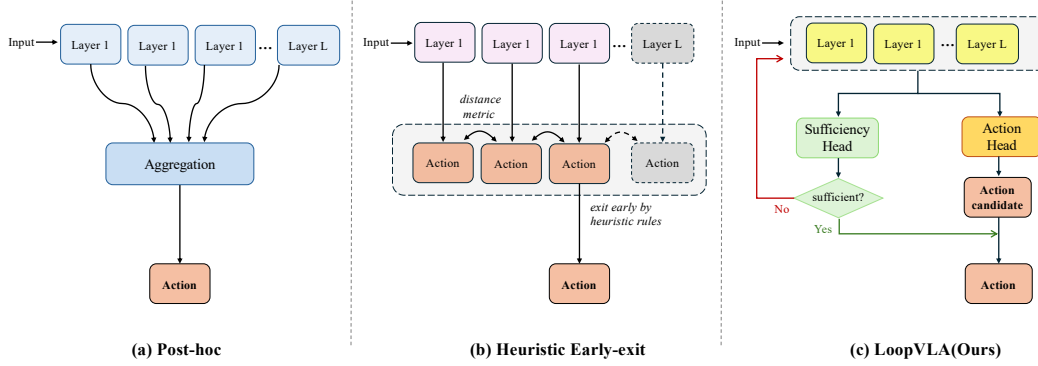


Figure 1: Comparison of intermediate-representation paradigms in VLA models. (a) Post-hoc methods aggregate or select intermediate features after full-depth computation. (b) Heuristic early-stop methods terminate computation using predefined criteria. (c) LoopVLA jointly models iterative refinement and sufficiency estimation through a shared recurrent Transformer.

2 Related Work

Vision-Language-Action Models. Vision-Language-Action (VLA) models have emerged as a promising paradigm for generalist robot manipulation by leveraging large-scale multimodal data and pretrained vision-language models (VLMs) [17, 18, 19]. A typical VLA system encodes visual observations and language instructions into latent representations using a VLM backbone, followed by a policy head that maps these representations to control actions [20].

Recent efforts have focused on scaling data, model capacity, and task diversity, leading to improved generalization across tasks and environments [21, 22]. In most existing approaches, action prediction is primarily based on the final-layer representations of the backbone, implicitly assuming that deeper representations are more suitable for decision making. While effective in practice, this design largely treats the output of the backbone as a fixed representation for downstream control.

Layer-wise Representation Readout. Prior work has shown that representations at different layers of VLMs encode information at varying levels of abstraction, from low-level perceptual cues to high-level semantic concepts [23]. To better exploit these intermediate representations, several studies explore layer-wise readout strategies for action prediction. One line of work attaches the action head to a fixed layer, assuming a single depth provides optimal features across inputs [14, 15]. While efficient, this design is inherently input-agnostic and may be suboptimal under varying task complexities.

Alternatively, multi-layer aggregation methods combine representations from different depths via attention or learned fusion modules [13, 12, 24]. Although more expressive, these approaches introduce additional computational overhead and training complexity. Despite their differences, both lines of work treat depth as a static architectural choice, overlooking the role of progressive refinement in shaping representations for accurate action prediction.

Iterative Refinement Transformer. Iterative formulations of Transformer architectures have been explored through recurrent designs, where a shared block is applied multiple times to progressively

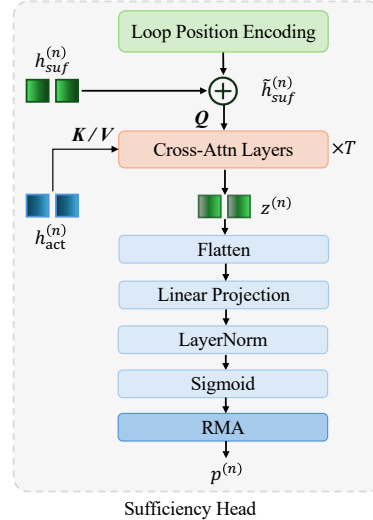


Figure 2: Sufficiency Head architecture. At iteration n , sufficiency tokens attend to action tokens through cross-attention layers with loop positional encoding to predict the halting probability $p^{(n)}$. RMA denotes the Remaining Mass Allocation mechanism.

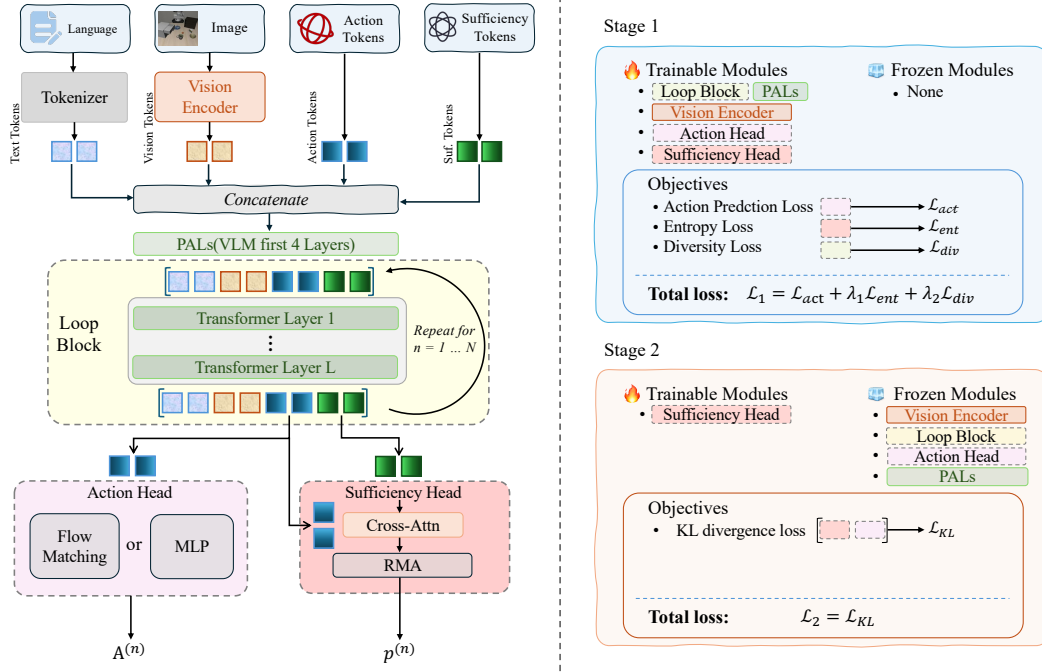


Figure 3: Overview of LoopVLA. Visual, language, action, and sufficiency tokens are processed by perceptual anchor layers followed by a shared recurrent Loop Block for iterative refinement. The action head predicts candidate actions at each iteration, while the sufficiency head estimates halting probabilities through cross-attention and remaining-mass allocation. Training is performed in two stages: joint refinement learning and sufficiency calibration.

refine representations. Prior works, such as Adaptive Computation Time and early exiting, regulate the number of refinement steps based on intermediate signals [25, 26], offering a view of depth as an unrolled iterative process rather than as a fixed architectural parameter.

Similar ideas have been explored in VLMs, where repeated refinement improves representation quality [27, 28]. However, these approaches are mainly studied in perception or generation settings. In contrast, VLA models require decision-oriented refinement, where intermediate representations must directly support action prediction, motivating a tighter coupling between refinement and action prediction.

3 Method

3.1 Problem Formulation

We consider a manipulation dataset with multiple tasks $\mathcal{D} = \{(\ell_i, \{(o_t, a_t)\}_{t=0}^{T_i})\}_{i=1}^N$. Each task i is specified by a language instruction ℓ_i and a sequence of demonstrations of length T_i . At each time step t , the observation $o_t \in \mathbb{R}^{V \times H \times W \times 3}$ comprises RGB images captured from V camera views, while the action $a \in \mathbb{R}^J$ denotes the corresponding control signal, where J is the number of degrees of freedom (DoF).

A typical VLA policy π_θ operates under a Markovian assumption, generating a sequence of future actions conditioned on the current observation and language instruction. The inputs are encoded by preserved vision-language model (VLM) layers into a latent representation,

$$h_t = \text{VLM}(o_t, \ell), \quad (1)$$

which is subsequently mapped to an action sequence by an action head,

$$A_{t:t+c} = \pi_\theta(h_t), \quad (2)$$

where $A_{t:t+c} = (a_{t+1}, \dots, a_{t+c}) \in \mathbb{R}^{c \times J}$ denotes c future actions. Instead of relying on a fixed latent representation, we formulate representation learning as a dynamic selection process as we proposed.

3.2 Model Architecture

We consider the LoopVLA setting, as illustrated in Fig. 3. We focus on a single-step formulation, where the policy operates on the current observation without explicit temporal dependencies. The policy receives visual observations, which are encoded into token representations via a vision encoder: $h_{\text{vis}} = \mathcal{E}_{\text{vis}}(o)$. Similarly, the language instruction is mapped into text tokens: $h_{\text{txt}} = \mathcal{E}_{\text{txt}}(\ell)$. We further introduce learnable action tokens $h_{\text{act}} \in \mathbb{R}^{N_{\text{act}} \times d}$ and sufficiency tokens $h_{\text{suf}} \in \mathbb{R}^{N_{\text{suf}} \times d}$, which are used to extract action features and loop-dependent representations for sufficiency estimation, respectively.

The tokens are concatenated along the sequence dimension: $h = [h_{\text{txt}}, h_{\text{vis}}, h_{\text{act}}, h_{\text{suf}}]$. To preserve the early-stage representations of the pretrained LLM, we retain the first four Transformer layers as Perceptual Anchor Layers (PALs), keeping their original architecture unchanged during iterative refinement. The resulting representation is computed as: $h^{(0)} = \text{PALs}(h)$, where the superscript 0 means the initial state after PALs.

Loop Block. We employ a shallow Transformer block as a recurrent module to iteratively refine representations. At iteration n , the hidden state is updated as

$$h^{(n)} = \text{LoopBlock}(h^{(n-1)}; M), \quad (3)$$

where LoopBlock denotes a stack of causal Transformer layers with shared parameters, and M is a structured causal attention mask that decouples action and sufficiency tokens.

Concretely, it follows standard causal masking, with sufficiency tokens fully visible to each other, while enforcing asymmetric visibility: action tokens attend to visual and language tokens, whereas sufficiency tokens attend to all tokens to aggregate global information.

Sufficiency Head. As illustrated in Fig. 2, we construct the sufficiency head from the refined representation at iteration n . We extract the sufficiency and action tokens, denoted as $h_{\text{suf}}^{(n)}$ and $h_{\text{act}}^{(n)}$, respectively. A loop-index positional encoding is added to the sufficiency tokens:

$$\tilde{h}_{\text{suf}}^{(n)} = h_{\text{suf}}^{(n)} + PE(n), \quad (4)$$

where $PE(n)$ denotes a sinusoidal positional encoding indexed by the iteration step, providing a loop-aware prior over refinement depth. It helps distinguish refinement stages and prevents collapse to trivial identity mappings.

The tokens are then linearly projected to form queries, keys, and values: $Q = W_q \tilde{h}_{\text{suf}}^{(n)}$, $K = W_k h_{\text{act}}^{(n)}$, $V = W_v h_{\text{act}}^{(n)}$. We apply a stack of cross-attention layers: $z^{(n)} = \text{CrossAttn}^{(T)}(Q, K, V)$, where $\text{CrossAttn}^{(T)}$ denotes T stacked cross-attention layers with shared parameters across iterations. Finally, an MLP followed by a sigmoid produces an intermediate halting score:

$$s^{(n)} = \sigma(\text{MLP}(z^{(n)})), \quad (5)$$

where the sigmoid function constrains the output to the range $(0, 1)$, enabling a probabilistic interpretation of the halting score.

Remaining Mass Allocation (RMA). To ensure proper normalization of sufficiency probabilities across loop iterations, we introduce a remaining mass allocation mechanism. Remaining mass is used to convert intermediate halting scores into a valid distribution over refinement steps. At each iteration, the halting probability is computed as

$$p^{(n)} = s^{(n)} \cdot r^{(n)}, \quad (6)$$

where $r^{(n)}$ denotes the remaining probability mass with an initial value $r^{(1)} = 1$, updated recursively by

$$r^{(n+1)} = r^{(n)} \cdot (1 - s^{(n)}). \quad (7)$$

This formulation ensures $\sum_{n=1}^N p^{(n)} \leq 1$ and enables adaptive halting across iterations.

Action Head. At iteration n , the action head maps the refined representation to an action chunk:

$$A^{(n)} = \pi_\theta(h^{(n)}), \quad (8)$$

where $A^{(n)} \in \mathbb{R}^{c \times J}$ denotes the sequence of c future actions as mentioned. The policy head π_θ can be instantiated with different parameterizations, such as an OFT-based head or a flow-matching head. Each iteration produces a candidate action, whose quality is assessed by the sufficiency head.

3.3 Dual-Stage Training

We adopt a dual-stage training strategy to learn both action prediction and sufficiency-aware halting behavior. The first stage jointly optimizes all modules to ensure strong action prediction performance while enabling the sufficiency head to learn a soft halting distribution.

Stage 1: Joint refinement learning. We jointly supervise all intermediate predictions to ensure each refinement step produces a meaningful action estimate:

$$\mathcal{L}_{\text{action}} = \sum_{n=1}^N \ell(A^{(n)}, \hat{A}), \quad (9)$$

where $\hat{A}$ denotes the ground-truth action and $\ell(\cdot, \cdot)$ denotes the ℓ_1 loss. This joint supervision stabilizes training and encourages progressively refined representations.

To further stabilize the halting behavior, we introduce two regularization terms during early training: an entropy regularization to prevent premature collapse, and a diversity regularization to discourage trivial identity mappings:

$$\mathcal{L}_{\text{ent}} = \sum_{n=1}^N p^{(n)} \log p^{(n)}, \quad \mathcal{L}_{\text{div}} = \sum_{i < j} \max(0, \cos(z^{(i)}, z^{(j)})). \quad (10)$$

The overall objective is

$$\mathcal{L}_{\text{stage1}} = \mathcal{L}_{\text{action}} + \lambda_1 \mathcal{L}_{\text{ent}} + \lambda_2 \mathcal{L}_{\text{div}}. \quad (11)$$

Stage 2: Sufficiency calibration. We freeze all modules except the sufficiency head and calibrate the halting distribution. This stage aims to decouple action learning from halting prediction, avoiding interference between the two objectives. It further sharpens the halting distribution by aligning it with the relative quality of intermediate predictions. A target distribution over refinement steps is defined as

$$q^{(n)} = \text{softmax}\left(-\ell(A^{(n)}, \hat{A})/\tau\right), \quad (12)$$

where $\tau > 0$ controls the sharpness. We align the predicted distribution $p^{(n)}$ with $q^{(n)}$ via

$$\mathcal{L}_{\text{stage2}} = \text{KL}(q \| p), \quad (13)$$

where $p = (p^{(1)}, \dots, p^{(N)})$.

3.4 Inference Recipe

At inference time, we utilize the learned halting distribution to select the final action, enabling a flexible trade-off between performance and efficiency.

Optimal selection. We run all N refinement steps and select the action with the highest probability as the optimal performance strategy :

$$n^* = \arg \max_n p^{(n)}, \quad A = A^{(n^*)}. \quad (14)$$

Adaptive trade-off. To reduce computation, we adopt an early stopping strategy based on the accumulated halting probability. At step n , refinement is terminated if either

$$\sum_{k=1}^n p^{(k)} \geq \theta \quad \text{or} \quad \max_{k \leq n} p^{(k)} \geq r^{(n+1)}, \quad (15)$$

Table 1: Main results on LIBERO across four task suites. ($L \otimes N$) denotes the loop configuration. Bold entries indicate our LoopVLA variants. Thrpt. denotes throughput (Hz), i.e., the average number of action predictions per second. For fairness, all methods are evaluated using eager attention without KV cache, except for methods marked with †. LoopVLA achieves consistent improvements in success rate while significantly reducing parameter count and improving throughput. Bold denotes the best result and underline denotes the second-best.

Model	Spatial	Object	Goal	Long	Average	Params ↓	FLOPs ↓	Thrpt. (Hz) ↑
Diffusion Policy [29]	78.3	92.5	68.3	50.0	72.4	-	-	-
OpenVLA [3]†	84.7	88.4	79.2	53.7	76.5	7.2B	6.58T	3.26
π_0 +FAST [30]	96.4	96.8	88.6	60.2	85.5	3.5B	606.68T	0.05
GR00T-N1.5 [15]	92.0	92.0	86.0	76.0	86.5	2.4B	0.47T	7.63
UniVLA [31]	96.5	96.8	95.6	92.0	95.2	7.2B	108.71T	1.41
π_0 [13]	<u>96.8</u>	98.8	95.8	85.2	94.1	4.0B	1.79T	3.70
$\pi_{0.5}$ [13]	<u>95.4</u>	98.4	<u>97.2</u>	89.6	95.1	4.1B	2.41T	3.11
F1-VLA [32]†	98.2	97.8	95.4	<u>91.3</u>	<u>95.7</u>	4.2B	5.93T	1.68
Qwen3FM	94.0	92.3	91.3	65.7	85.8	2.3B	0.53T	0.97
LoopFM α ($3 \otimes 8$)	91.0	99.0	95.3	79.0	91.1	<u>1.3B</u>	<u>0.38T</u>	2.04
Qwen3OFT	95.0	97.0	97.1	90.5	94.9	2.2B	0.53T	10.49
LoopOFT $_0$ ($3 \otimes 8$)	93.4	98.2	96.0	90.6	94.6	1.2B	0.31T	18.41
LoopOFT α ($3 \otimes 8$)	94.0	<u>99.6</u>	96.8	91.0	95.3	1.2B	0.42T	<u>12.15</u>
LoopOFT* ($3 \otimes 8$)	95.0	100	97.4	91.0	96.0	1.2B	0.53T	10.93

Table 2: Zero-shot results on LIBERO-Plus across multiple generalization factors. LoopOFT α maintains competitive performance with significantly fewer parameters, highlighting strong generalization and efficiency.

Model	Camera	Robot	Language	Light	Background	Noise	Layout	Total	Params↓
OpenVLA [3]	0.8	3.5	23.0	8.1	34.8	15.2	28.5	15.6	7.2B
UniVLA [22]	1.8	46.2	69.6	69.0	81.0	21.2	31.9	45.8	7.2B
π_0 [13]	13.8	6.0	58.8	85.0	81.4	79.0	68.9	53.6	4.0B
π_0 +FAST [30]	<u>65.1</u>	21.6	61.0	73.2	73.2	<u>74.4</u>	68.8	61.6	3.5B
$\pi_{0.5}$ [33]	53.0	<u>50.3</u>	65.7	83.1	77.3	53.2	72.7	65.0	4.1B
VLA-Adapter [34]	76.6	36.4	73.8	71.0	70.2	37.4	57.2	60.4	1.2B
Qwen3OFT	45.6	55.0	<u>73.5</u>	86.1	90.2	69.3	<u>72.4</u>	68.8	<u>2.2B</u>
LoopOFT α ($3 \otimes 8$)	58.3	41.7	66.7	88.9	<u>88.3</u>	61.4	71.8	<u>65.8</u>	1.2B

where θ is a predefined threshold determined heuristically based on the $w\sigma$ rule of a normal distribution (e.g., $\theta \approx 0.68, 0.95, 0.997$ for $w = 1, 2, 3$, respectively). The final action is selected from the most probable visited step:

$$n^* = \arg \max_{k \leq n} p^{(k)}, \quad A = A^{(n^*)}. \quad (16)$$

4 Experiments

4.1 Experiment Settings

Simulation Benchmark Details. We evaluate on LIBERO [35], VLA-Arena [36], and LIBERO-Plus [37]. LIBERO contains multiple task suites covering spatial reasoning, object interaction, and long-horizon manipulation, while VLA-Arena provides a standardized benchmark with diverse tasks and difficulty levels. To assess generalization, we additionally report zero-shot performance on LIBERO-Plus, which introduces unseen task compositions.

Following prior work, we report the average success rate over 50 trials per suite on LIBERO and the average success rate across all tasks on VLA-Arena. For LIBERO, models are jointly trained on the combined data from all four task suites and evaluated on each suite individually using the same checkpoint. For VLA-Arena, training uses only the VLA-Arena-L0-Small subset. For LIBERO-Plus, we report zero-shot success rates without additional finetuning.

Baselines. Our experiments are instantiated on a VLA architecture built upon Qwen3VL-2B-Instruction [19], with a SigLIP-2 [38] visual encoder and a 28-layer Transformer decoder, including

Table 3: Results on VLA-Arena (L0 level) across four task categories. SmolVLA is trained on VLA-Arena-L0-Large, while the other methods are trained on VLA-Arena-L0-Small.

Method	Safety	Distractor	Extrapolation	Long Horizon	Average
SmolVLA [12]	33.0	48.0	23.0	74.0	37.0
Qwen3OFT	54.0	71.0	13.3	59.0	47.0
LoopVLA	55.6	60.0	12.0	76.0	48.7

a 3-layer DeepStack module. We freeze the first four layers together with the DeepStack module—referred to as the *PALs*—to preserve low-level perceptual features, and apply looping only to the remaining 24 layers. This confines iterative refinement to higher-level representations where sufficiency is most relevant while maintaining stable early-stage features.

For action prediction, we adopt two main stream policy heads: an OFT head (OpenVLA-OFT [39]) and a flow-matching head (GR00T [15]), both kept in their original minimal forms. The primary experimental validation is conducted using the StarVLA framework [40].

Implementation Details. For LoopVLA, we denote a configuration by $(L \otimes N)$, where L is the number of retained Transformer layers and N is the maximum number of loop iterations. All models are initialized from pretrained checkpoints, retaining only the first k layers of the LLM decoder while discarding the rest.

For all simulation benchmarks, we perform full-parameter finetuning unless otherwise specified. Training is conducted on NVIDIA A100 GPUs, while inference is performed on a single NVIDIA RTX 4090 GPU.

For result annotation, * indicates the optimal selection inference configuration, while subscript α denotes a trade-off inference strategy balancing performance and efficiency. A subscript with a number (e.g., 1) represents inference with a fixed number of loops.

4.2 Main Results on Simulation Environment

Results on LIBERO. We evaluate LoopVLA on LIBERO under a unified setup with identical backbones, using a $(3 \otimes 8)$ configuration. As shown in Table 1, LoopVLA achieves strong performance while improving efficiency. LoopOFT* attains an average success rate of **96.0%**, outperforming strong baselines such as π_0 and Qwen3OFT, despite using fewer parameters. Some baseline results are reported from or reproduced following VLA-Evaluation-Harness. [41].

LoopVLA also improves performance on challenging settings, reaching **91.0%** on Long-horizon tasks, while maintaining near-saturated results on Object tasks. The trade-off variant (LoopOFT $_{\alpha}$) achieves **95.3%**, indicating robustness under reduced computation. Compared to Qwen3OFT, our method significantly reduces model size while maintaining performance, and supports flexible performance–efficiency trade-offs across inference strategies. These gains are attributed to the iterative refinement design, which adaptively allocates computation across steps.

Results on VLA-Arena. We evaluate LoopVLA on the VLA-Arena benchmark under the same experimental protocol as LIBERO. Results are reported in Table 3.

LoopVLA outperforms Qwen3OFT in average performance, with gains on Safety and Long Horizon tasks. Despite minor variations in other categories, the results indicate that LoopVLA generalizes effectively beyond LIBERO, while remaining efficient under the same training setup.

Generalization on LIBERO-Plus. We evaluate zero-shot robustness on LIBERO-Plus. As shown in Table 2, LoopVLA achieves **66.5%** success, remaining competitive with larger models such as $\pi_{0.5}$ (65.0%) while using fewer parameters, with a modest $\sim 3\%$ gap to the strongest baseline.

LoopVLA remains robust across variations, particularly under visual changes (e.g., lighting and background), though performance degrades on more challenging factors (e.g., robot variation). This trade-off reflects reduced model capacity and adaptive computation, suggesting that iterative refinement maintains stable generalization under distribution shifts with improved efficiency.

Table 4: Performance impact of loop configurations (Stage 1 training only).

Config	S	O	G	L	Avg	Params↓
$(8 \otimes 3)$	95.0	100	95.3	80.7	92.8	1.48B
$(6 \otimes 4)$	96.0	98.7	95.7	85.0	93.9	1.37B
$(4 \otimes 6)$	96.4	96.2	97.0	90.5	95.0	1.27B
$(3 \otimes 8)$	94.7	98.6	96.6	90.2	95.0	1.22B
$(2 \otimes 12)$	91.2	98.6	95.0	83.4	92.1	1.17B

Table 5: Impact of inference layer settings (both Stage 1 and 2 training).

Setting	S	O	G	L	Avg
$(3 \otimes 8)_1$	91.0	98.7	95.4	90.0	93.8
$(3 \otimes 8)_3$	93.2	99.4	96.0	90.6	94.9
$(3 \otimes 8)_5$	94.6	96.7	96.2	91.0	94.7
$(3 \otimes 8)_6$	93.4	99.4	95.6	90.6	94.8
$(3 \otimes 8)_8$	92.0	99.0	94.6	91.0	94.2
$(3 \otimes 8)^*$	95.0	100	97.4	91.0	96.0

Table 6: Ablation of the sufficiency head on LIBERO, comparing a standard MLP against the proposed sufficiency-aware design.

Method	Spatial	Object	Goal	Long	Average
Direct MLP	94.0	99.2	97.2	87.0	94.4
Sufficiency Head	95.0	100.0	97.4	91.0	96.0

4.3 Ablation Study

Effect of Loop Configurations. We study how to allocate a fixed computation budget by varying loop configurations ($L \otimes N$) under $L \times N = 24$. As shown in Table 4, performance follows a non-monotonic trend. Here, S, O, G, L, and Avg denote the Spatial, Object, Goal, Long, and average task suites, respectively. Balanced configurations (e.g., $(4 \otimes 6)$ and $(3 \otimes 8)$) achieve the best results, while more extreme settings degrade performance. This suggests that neither increasing depth nor iteration alone is sufficient; instead, effective performance arises from a proper balance between the two. We adopt $(3 \otimes 8)$ as the default configuration.

Inference Layer Selection. Under a fixed $(3 \otimes 8)$ configuration, we evaluate different inference layer selections (Table 5). Performance again exhibits a non-monotonic pattern: both shallow and overly deep selections underperform, while intermediate layers yield better results. Notably, adaptive selection (*) consistently outperforms fixed strategies, indicating that the optimal computation depth is input-dependent. The variation of optimal depths across tasks further supports the effectiveness of Stage 2 training. We further visualize the distribution of maximum sufficiency across different LIBERO task suites in Appendix Sec. A.3, highlighting task-dependent variation in refinement behavior.

Effect of the Sufficiency Head. We replace the proposed sufficiency head with a standard MLP. As shown in Table 6, this leads to a consistent performance drop, especially on long-horizon tasks. This suggests that a simple MLP is insufficient to capture the notion of representational sufficiency. In contrast, the proposed design explicitly models interactions between sufficiency and action tokens, enabling more accurate sufficiency estimation and leading to improved overall performance.

5 Conclusion

In this work, we revisit VLA policy learning from the perspective of representational sufficiency. Rather than relying on fixed-depth representations or heuristic stopping criteria, we formulate action prediction as an iterative refinement process in which representation quality and decision sufficiency are jointly modeled.

Based on this formulation, we propose LoopVLA, a recurrent VLA architecture with a shared looped Transformer, a dual-head design, and a distribution alignment objective for sufficiency learning. By decoupling representation refinement from absolute layer depth, LoopVLA achieves a substantially improved efficiency–performance trade-off, reducing model parameters while maintaining or improving policy performance across multiple embodied benchmarks.

We hope this work highlights the importance of adaptive representation refinement in embodied policy learning and motivates future research on scalable, content-aware, and computation-efficient VLA architectures.

References

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, 2023.
- [2] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.
- [4] Oier Mees, Dibya Ghosh, Karl Pertsch, Kevin Black, Homer Rich Walke, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, et al. Octo: An open-source generalist robot policy. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.
- [5] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [6] Xi Chen, Xiao Wang, Soravit Changpinyo, Anthony J Piergiovanni, Piotr Padlewski, Daniel Salz, Sebastian Goodman, Adam Grycner, Basil Mustafa, Lucas Beyer, et al. Pali: A jointly-scaled multilingual language-image model. *arXiv preprint arXiv:2209.06794*, 2022.
- [7] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- [8] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.
- [9] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [10] Isaac L Kurtzer, J Andrew Pruszynski, and Stephen H Scott. Long-latency reflexes of the human arm reflect an internal model of limb dynamics. *Current Biology*, 18(6):449–453, 2008.
- [11] J Andrew Pruszynski and Stephen H Scott. Optimal feedback control and the long-latency stretch response. *Experimental brain research*, 218(3):341–359, 2012.
- [12] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [13] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. *Proceedings of Robotics: Science and Systems*, 2025.
- [14] Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdiñç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-flow models. In *Conference on Robot Learning*, pages 3736–3761. PMLR, 2025.

- [15] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Jim Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [16] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: dynamic inference of multimodal large language models for efficient robot execution. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, pages 56619–56643, 2024.
- [17] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *International Conference on Machine Learning*, pages 23123–23144. PMLR, 2024.
- [18] Andreas Steiner, André Susano Pinto, Michael Tschanen, Daniel Keysers, Xiao Wang, Yonatan Bitton, Alexey Gritsenko, Matthias Minderer, Anthony Sherbondy, Shangbang Long, et al. Paligemma 2: A family of versatile vlms for transfer. *arXiv preprint arXiv:2412.03555*, 2024.
- [19] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [20] Kento Kawaharazuka, Jihoon Oh, Jun Yamada, Ingmar Posner, and Yuke Zhu. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 2025.
- [21] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.
- [22] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Learning to act anywhere with task-centric latent actions. In *Proceedings of Robotics: Science and Systems*, 2025.
- [23] Haoran Chen, Junyan Lin, Xinhao Chen, Yue Fan, Xin Jin, Hui Su, Jianfeng Dong, Jinlan Fu, and Xiaoyu Shen. Rethinking visual layer selection in multimodal llms. *arXiv e-prints*, pages arXiv-2504, 2025.
- [24] Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, et al. Internvla-m1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025.
- [25] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [26] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [27] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [28] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.
- [29] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

- [30] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [31] Shengliang Deng, Mi Yan, Songlin Wei, Haixin Ma, Yuxin Yang, Jiayi Chen, Zhiqi Zhang, Taoyu Yang, Xuheng Zhang, Heming Cui, et al. Graspvla: a grasping foundation model pre-trained on billion-scale synthetic action data. In *Conference on Robot Learning*, pages 1004–1029. PMLR, 2025.
- [32] Qi Lv, Weijie Kong, Hao Li, Jia Zeng, Zherui Qiu, Delin Qu, Haoming Song, Qizhi Chen, Xiang Deng, and Jiangmiao Pang. F1: A vision-language-action model bridging understanding and generation to actions. *arXiv preprint arXiv:2509.06951*, 2025.
- [33] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 27–30 Sep 2025.
- [34] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, et al. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646, 2026.
- [35] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [36] Borong Zhang, Jiahao Li, Jiachen Shen, Yishuai Cai, Yuhao Zhang, Yuanpei Chen, Juntao Dai, Jiaming Ji, and Yaodong Yang. Vla-arena: An open-source framework for benchmarking vision-language-action models. *arXiv preprint arXiv:2512.22539*, 2025.
- [37] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [38] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.
- [39] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Proceedings of Robotics: Science and Systems*, 2025.
- [40] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.
- [41] Suhwan Choi, Yunsung Lee, Yubeen Park, Chris Dongjoo Kim, Ranjay Krishna, Dieter Fox, and Youngjae Yu. vla-eval: A unified evaluation harness for vision-language-action models. *arXiv preprint arXiv:2603.13966*, 2026.

A Appendix

A.1 LIBERO-Plus Benchmark

We evaluate the generalization capability of LoopVLA on LIBERO-Plus, a benchmark designed for in-depth robustness analysis of Vision-Language-Action models under controlled distribution shifts.

LIBERO-Plus performs a systematic evaluation by introducing perturbations along seven dimensions: object layout, camera viewpoints, robot initial states, language instructions, lighting conditions, background textures, and sensor noise. These factors cover a wide spectrum of variations, ranging from low-level visual changes (e.g., lighting, background, noise) to higher-level variations in scene composition, viewpoint, and instruction semantics. Representative examples are shown in Figure 4.

All evaluations are conducted in a zero-shot setting without any additional fine-tuning, following the official evaluation protocol. Performance is measured by the average task success rate across tasks within each perturbation dimension, as well as the overall average across all dimensions. This setup enables a comprehensive assessment of model robustness and reliability under realistic variations beyond the standard LIBERO benchmark.

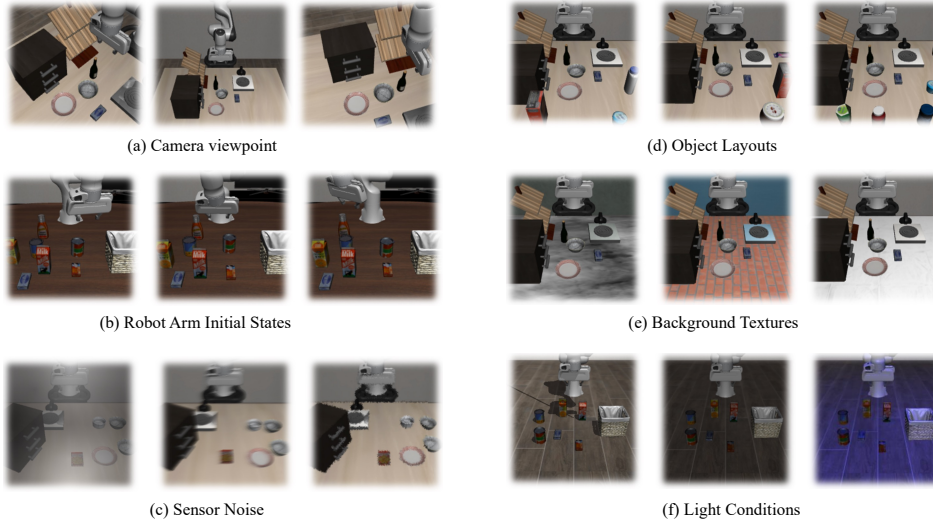


Figure 4: Examples of distribution shifts in LIBERO-Plus across different generalization dimensions: (a) Camera viewpoints, varying the observation perspective; (b) Robot arm initial states, changing the robot’s starting configuration; (c) Sensor noise, introducing visual perturbations and corruption; (d) Object layouts, altering spatial arrangements of objects; (e) Background textures, modifying scene appearance; (f) Light conditions, varying illumination settings. These controlled perturbations evaluate robustness from low-level visual variations to higher-level spatial and embodiment-related changes.

A.2 VLA-Arena Benchmark

We further evaluate LoopVLA on VLA-Arena, an open-source benchmark for systematic evaluation of Vision-Language-Action (VLA) models. VLA-Arena provides a full evaluation pipeline, including scene modeling, demonstration collection, model training, and standardized evaluation.

The benchmark consists of 170 tasks organized into 11 specialized task suites, with hierarchical difficulty levels ranging from L0 to L2. It is designed to assess model performance across four key domains: (i) safety, requiring reliable and safe interaction with the environment; (ii) distractors, evaluating robustness to environmental variability; (iii) extrapolation, measuring generalization to novel task compositions; and (iv) long-horizon reasoning, testing the ability to execute extended action sequences. Example tasks are illustrated in Figure 5.

In our experiments, we focus on the L0 setting and use the VLA-Arena-L0-Small subset for both training and evaluation. All methods are evaluated under a unified protocol with consistent inference

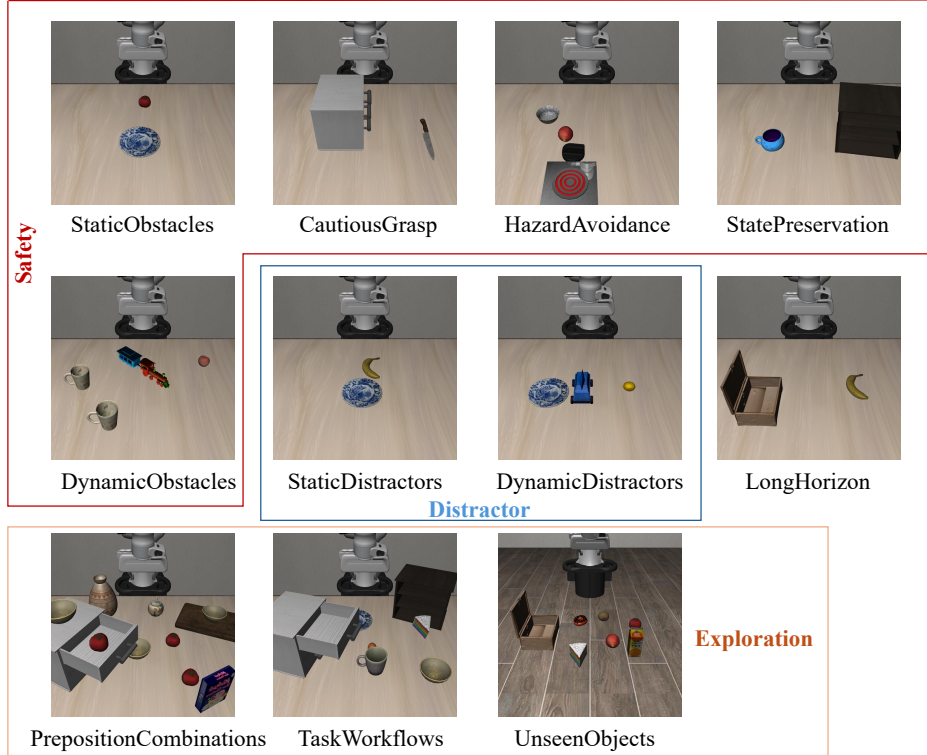


Figure 5: Representative L0 task suites in VLA-Arena across four domains: safety (top), distractors (middle), extrapolation (bottom), and long-horizon tasks (right). These tasks evaluate reliability, robustness, generalization, and long-horizon reasoning.

settings and success metrics. Performance is reported using the average task success rate across all task suites following the official benchmark setup.

A.3 Distribution of Optimal Loop Indices

We analyze the distribution of selected loop indices n^* across different LIBERO task suites, as shown in Fig. 6. The statistics are obtained from uniformly randomly sampled episodes within each suite, rather than exhaustively evaluating the full benchmark.

A clear distinction emerges across task types. Simpler suites such as *Object* show a strong concentration on early loop indices (e.g., $n = 2$), indicating that minimal refinement is sufficient. In contrast, more complex suites such as *Spatial* and *Goal* exhibit distributions skewed toward larger loop indices (e.g., $n = 7, 8$), suggesting a greater need for iterative refinement. The *Long* suite presents a mixed pattern, with both early stopping and deeper refinement occurring frequently, reflecting the variability of long-horizon tasks. This also explains the low frequency of intermediate loop indices (e.g., $n = 3, 4$), as the model tends to either terminate early when sufficient information is obtained or continue refinement toward later steps for more complex decisions, resulting in a bimodal preference.

Overall, these results support that LoopVLA dynamically adjusts its inference depth according to task difficulty, rather than relying on a fixed number of refinement steps.

A.4 Training Hyperparameters

We follow the training and inference setup described in Sec. 3.3 and Sec. 3.4. All hyperparameters are summarized in Table 7.

In brief, models are trained using AdamW with a cosine learning rate schedule and a global batch size of 64. Training of stage 2 is typically updated for 10K steps, or equivalently one epoch over the

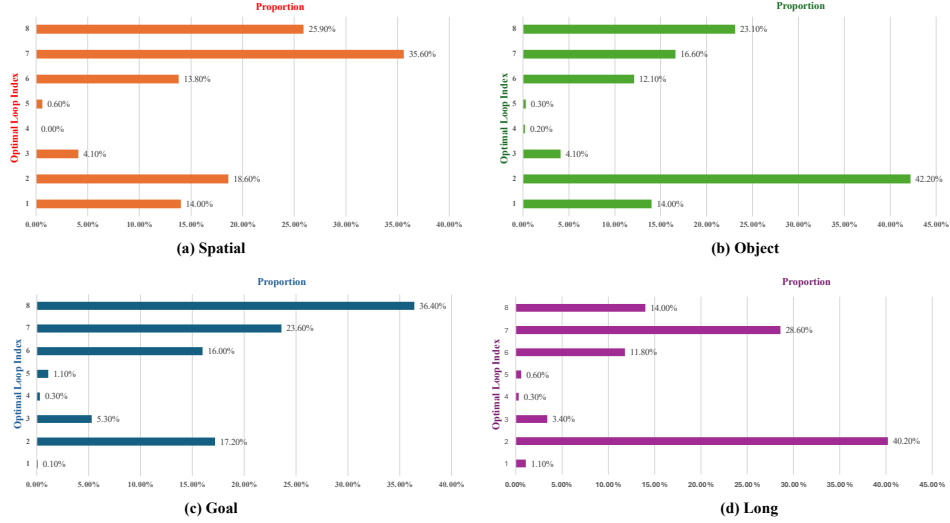


Figure 6: Distribution of selected loop indices n^* across different LIBERO task suites, computed from sampled evaluation episodes. Different task suites exhibit distinct distribution patterns over loop indices, reflecting varying computational demands.

Table 7: Training and inference hyperparameters.

Hyperparameter	Value
<i>Training</i>	
Optimizer	AdamW
LR Scheduler	Cosine
Global Batch Size	64
Action Chunk Size	8
Action Tokens	8
Sufficiency Tokens	3
Entropy Reg. λ_1	0.001
Diversity Reg. λ_2	0.01
Warmup Steps	1K
Temperature τ	0.5
<i>Inference</i>	
Threshold θ	0.68

dataset. Entropy and diversity regularization are applied during early training to stabilize the halting behavior.

All experiments are implemented in PyTorch and trained on 8 NVIDIA A100 (40GB) GPUs. For fair comparison, all methods use the same backbone architectures and data protocols as their corresponding baselines.

A.5 Limitations

Although LoopVLA improves the efficiency-performance trade-off of VLA models, several limitations remain. First, while the proposed recurrent refinement mechanism demonstrates promising robustness and generalization ability, handling more severe distribution shifts and embodiment variations may require additional scaling in model capacity and training data.

Second, due to computational resource constraints, the current study does not extensively explore scaling behaviors across larger model sizes and data scales. A more systematic investigation of scaling properties may further reveal the potential and limitations of the proposed framework.

Finally, the current study primarily focuses on simulation benchmarks, and future work will extend the framework to real-world robotic platforms and broader embodied manipulation settings.